Open-Source Toolkit for Your Multi-Platform ERP

Research pass across GitHub, vendor docs, and dev forums, mapped to each module you listed. Everything here is free/open source with a real API or readable source — the kind of thing you can point Cursor at and say "clone this, read the schema, extend it," rather than something Cursor has to build from a blank file.

Recommended core architecture

- **Backend of record:** ERPNext (Frappe Framework) for Financial, HR, Payroll, Inventory, and Procurement. All five live in one GPLv3 codebase on one database, so nothing needs syncing between separate accounting/HR/inventory tools — that's the single biggest integration win available to you.
- **POS + storefronts:** two real paths exist (stay inside ERPNext vs. put a dedicated commerce platform in front). Laid out in §8 — worth deciding before any mobile app code gets written, since it determines what the apps talk to.

- **Satellite services**, each doing one job over its own API: Traccar (GPS), Casbin (permissions), Meilisearch (search), Gorse + Metabase/Superset (analytics), Hyperledger Fabric or a hash-chain (blockchain stock ledger).

Quick reference

Module	Primary pick	License
Financial Management	ERPNext – Accounting	GPLv3
HR & Onboarding	Frappe HR	GPLv3
Payroll	Frappe HR – Payroll	GPLv3
Identity Mgmt (QR ID cards)	`qrcode` / `node-qrcode` + ZXing	MIT / Apache-2.0
Inventory + e-approvals	ERPNext – Stock	GPLv3
Blockchain stock ledger	hash-chain (custom) or Hyperledger Fabric	— / Apache-2.0
Procurement	ERPNext – Buying	GPLv3

Module	Primary pick	License
Logistics / GPS	Traccar (+ OSRM for routing)	Apache-2.0 / BSD
POS + Storefront	ERPNext POS, or Bagisto, or Medusa	GPLv3 / MIT / MIT
Parts search index	Meilisearch	MIT
VIN decode	NHTSA vPIC API	Free, public
Cross-sell / analytics	Gorse + Metabase or Superset	Apache-2.0 / AGPLv3 or Apache-2.0
RBAC (MPM Companion)	Casbin	Apache-2.0

---

1. Financial Management

ERPNext – Accounting module — `github.com/frappe/erpnext` — GPLv3

Double-entry accounting, multi-currency, bank reconciliation, and on-demand P&L / balance sheet /

cash-flow generation are native, not bolt-ons. Cursor task: use ERPNext's REST API (`/api/resource/...`) and the Frappe "Report" doctype to add any statement formats beyond the defaults.

2. Human Resources

Frappe HR — `github.com/frappe/hrms` — GPLv3

Built by the same team specifically as ERPNext's HR companion — installs alongside it and shares the database, so employee records don't get duplicated between HR and Accounting. Covers the full employee lifecycle including onboarding, leave/attendance with geolocated check-in, and ships its own mobile PWA. Cursor task: onboarding workflow changes happen in Frappe's no-code Workflow builder — have Cursor generate the workflow JSON rather than hand-building UI.

3. Payroll Processing

Frappe HR – Payroll (same repo as above) — GPLv3

Salary structures, payslips, payroll entries, and regional tax-slab logic ship built in (India's rules are most complete out of the box — plan to have Cursor localize the tax logic for your jurisdiction). Note on Odoo, for comparison: Odoo's native Payroll app is Enterprise-only (paid); Community edition users patch

it in with a third-party module, which is a less integrated path than what Frappe HR gives natively.

4. Identity Management (QR-coded ID cards)

Less "install a product," more wiring two small pieces together:

- **Generate:** `qrcode` (Python) or `node-qrcode` (Node) — both MIT — encode each employee ID into a QR image.
- **Design the badge:** Frappe's built-in Print Format designer can lay a badge template (photo, name, QR) directly off the Employee doctype, so it's always in sync with HR records. A free reference implementation to adapt: SourceCodester's "Employee ID Card Generator with QR Code" (Django) shows this exact pattern end to end.
- **Scan/validate:** ZXing (Apache-2.0, Android-native) or Google's ML Kit Barcode Scanning (free, on-device) for checkpoints and attendance readers — the same scanning approach as the POS app in §8.

5. Inventory Management

ERPNext – Stock module (same repo/license as §1)

Warehouses, stock ledger, batch/serial tracking. Electronic approval workflows are a built-in feature (Frappe's Workflow engine) — no separate approval tool needed.

Blockchain-backed stock movement — worth being deliberate about which one you actually need:

- **Real distributed ledger:** Hyperledger Fabric (Linux Foundation, Apache-2.0) — a genuine permissioned blockchain. Justified if independent parties (franchise partners, external auditors, other warehouses that don't trust each other's databases) need to agree on stock state without one of them holding the source of truth. A curated list of community reference supply-chain chaincode apps exists at `github.com/wearetheledger/awesome-hyperledger-fabric` — useful as a starting skeleton, though none are production-ready as-is.
- **Lightweight tamper-evidence** (what most commercial "blockchain inventory" products actually ship under the hood): have Cursor add a `previous_hash` / `entry_hash` field to ERPNext's Stock Ledger Entry, computed as SHA-256(previous entry's hash + this entry's data) on write. Same tamper-evident guarantee — anyone can prove the history wasn't edited after the fact — without operating a distributed ledger network.

- Honest read: unless genuinely independent parties need to trust the ledger without trusting *you*, start with the hash-chain. It's a day of schema work, not a new system to run.

6. Supply Chain & Procurement

ERPNext – Buying module — RFQs, supplier quotations, purchase orders live in the same database as Inventory and Accounting, so a PO automatically ties to the stock it receives and the AP entry it generates. Nothing separate to integrate.

7. Logistics Support (live GPS tracking)

Traccar — `github.com/traccar/traccar` — Apache-2.0

Self-hosted GPS platform: 200+ device protocols, real-time REST/WebSocket position feed, geofencing, alerts. For a delivery fleet, the companion Traccar Client app turns a driver's own phone into the GPS tracker — no hardware purchase needed to get started.

OSRM (Open Source Routing Machine, BSD) pairs well with Traccar if you want route optimization / ETA calculation for dispatch, not just a dot on a map.

Cursor task: the "assign driver → track delivery → mark complete" dispatch screen is custom work built against Traccar's API — Traccar gives you the position data, not a dispatch UI.

8. Omnichannel E-commerce & POS

The module with the most viable paths. Pick based on how much you value one source of truth vs. a more polished storefront out of the box.

Path A — stay inside the ERPNext ecosystem (tightest integration):

- ERPNext's own POS/Selling doctypes already support barcode fields.
- Add camera QR scanning with `react-native-vision-camera` + the `react-native-vision-camera-barcode-scanner` plugin (MLKit-based, actively maintained). Avoid `react-native-qrcode-scanner` — it's abandoned.
- Worth evaluating: **TailPOS** (`github.com/bailabs/tailpos`) — an offline-first mobile POS built specifically to two-way-sync with ERPNext, already using a tablet/phone's rear camera as the scanner. Check current commit activity before committing to it — community ERPNext add-ons vary in maintenance level.

Path B — dedicated commerce platform in front of ERPNext/Odoo for the back office:

- **Bagisto** — `github.com/bagisto/bagisto` — MIT (fully permissive, no copyleft to plan around). Laravel + Vue, with a native POS module (real-time sync between physical and online stock) and an official B2B suite (`github.com/bagisto/b2b-suite`: company accounts, RFQ, requisition lists, credit limits, sales-rep assignment) — this covers your tiered mechanic pricing directly. Headless/GraphQL mode with a Next.js storefront starter if you want one API feeding web + native apps.
- **Medusa.js** — `medusajs.com` — MIT. Node/TypeScript, API-first. The official free B2B starter ships company accounts, spend limits, cart-approval flows, and per-account price lists — again a direct fit for mechanic tiered pricing — plus built-in returns/exchange workflows. Comes with an official Next.js storefront; mobile apps call the same Store API.
- **Vendure** is a third option if you'd rather have GraphQL end to end — its core is GPLv3 with paid enterprise add-ons (open-core model), unlike Medusa's fully-permissive MIT, worth factoring in.

Either path, cart QR-scanning on the sales associate's phone is the same `react-native-vision-camera`

approach — just calling Bagisto's or Medusa's cart API instead of ERPNext's.

9. Visual Parts Catalog (motor vehicle spares)

Straight talk: nothing free and open source does this end-to-end. What exists are strong building blocks you assemble:

- **VIN search:** NHTSA vPIC API (`vpic.nhtsa.dot.gov`) — free, no key, no registration, official US DOT data. JS (`nhtsa-api-wrapper`) and Python (`vpic-api`) wrappers exist. Strongest for US-market vehicles — if you're stocking non-US-spec vehicles, coverage thins out and you'll lean more on your own fitment table.
- **Year/Make/Model search:** there's no free equivalent of the industry's real fitment database. The Auto Care Association's ACES (fitment) and PIES (product data) are the actual industry *standards*, but the reference databases behind them (VCdb, PCdb, Qdb, PAdb) require a paid Auto Care Association subscription. What's free is the standard's structure — designing your own fitment table to match ACES/PIES field names now means you can cleanly import real supplier data feeds later without a rebuild.

- **Part number / keyword search:** Meilisearch (MIT, fully free self-hosted) — typo-tolerant instant search, and its hybrid keyword+semantic search (local embeddings, no external API required) is the most honest way to deliver "search by describing the part" rather than exact-match only. Typesense (GPLv3) is a solid alternative if you prefer its specific feature set.
- **The "4-way search" itself** is a thin routing layer Cursor builds: input looks like a 17-character VIN → vPIC; looks like a part number → exact lookup; otherwise → Meilisearch full-text + your fitment table. No single product does this — it's assembly, not installation.
- **"AI-driven analytics":** Gorse (`github.com/gorse-io/gorse`, open source, Go) for genuine ML-driven cross-sell — "customers who bought this also bought," "commonly replaced together" — sitting on top of the catalog, plus Metabase or Apache Superset (both open source; Metabase's free edition is AGPLv3, Superset is Apache-2.0) wired to the same database for demand/trend dashboards.

10. Automated returns & tiered B2B pricing (cross-cutting)

Both are configuration, not new tools — every commerce backend above already has them:

- ERPNext: Sales Return / credit notes; Pricing Rules keyed to customer group.
- Odoo: Returns wizard; Pricelists per customer segment.
- Medusa / Bagisto B2B: covered natively, as noted in §8.

Have Cursor build automation on top (auto-approve returns under a value threshold, auto-tag mechanics into the right price tier) rather than building return/pricing logic from scratch.

11. MPM Companion — RBAC + Notice Board

RBAC: Casbin — `github.com/apache/casbin` (an Apache Software Foundation project) — Apache-2.0

Handles RBAC/ABAC/ACL with the same API across roughly 15 languages/frameworks; policies can live in your existing database. One of the most widely used authorization libraries there is — low-risk pick. Pair it with an identity/login layer if you want single sign-on across the ERP, storefronts, and MPM Companion — Keycloak (Apache-2.0) is the standard open-source choice for that; Casbin then handles "what can this

logged-in user do" on top of Keycloak's "who is this user."

Notice Board: small enough that a dedicated project is probably overkill — a CRUD announcements table + push notification is a Cursor afternoon, not a library search. If you want more than a basic feed (rich text, comments, read receipts), Wiki.js or Outline (both open source) repurpose well as an internal announcements hub.

License notes worth keeping in mind

MIT / Apache-2.0 (Bagisto, Medusa, Traccar, Casbin, Meilisearch, OSRM) are permissive — no obligation to release your own code. GPLv3 (ERPNext, Frappe HR, Vendure's core, Typesense) requires that if you distribute *modified copies of that specific codebase*, the modifications stay open — this generally doesn't affect you running it internally for your own store. AGPLv3 (Metabase's free edition) extends that obligation to running it as a network service — worth a closer read if external parties ever interact with a modified instance directly, but not a concern for an internal dashboard. None of this is legal advice — worth a real read of each license if you're unsure how it applies to your situation.

Suggested sequence for Cursor

1. Stand up ERPNext + Frappe HR via Docker (`frappe_docker` on GitHub) first — gets Financial/HR/Payroll/Inventory/Procurement running as one system before anything else is built.
2. Point Cursor at the Frappe doctype JSON schemas to generate custom fields/workflows (QR ID field, hash-chain field on Stock Ledger Entry, etc.) instead of hand-writing them.
3. Decide Path A vs. Path B in §8 before writing storefront code — it determines what the mobile apps talk to.
4. Build the satellite services (Traccar, Casbin, Meilisearch, Gorse) as separate deployments called over their REST/GraphQL APIs rather than folded into the ERPNext codebase — keeps each one's upgrades independent.
5. Budget the parts catalog (§9) as a real build. It's the one module here without a shortcut.